

在庫計算機能仕様書

概要

生産計画と実績に基づいて、在庫・計画在庫を自動計算する機能。仕損数の集計、実績出庫数の計算、時制を考慮した在庫推移計算を行う。

関連テーブル

LineBacklog（ライン別バックログ）

生産計画と在庫情報を保持するメインテーブル。

フィールド名	型	説明
plan_date	DATE	計画日
process	FK	工程
product	FK	製品
line	FK	ライン
order_qty	INT	後工程需要（内示/計画）
plan_qty	INT	計画生産数
actual_qty	INT	実績生産数
stock_qty	INT	実在庫
planned_stock_qty	INT	計画在庫
adjust_qty	INT	調整数（手動調整、棚卸差異）
scrap_qty	INT	仕損数（自動集計）
actual_shipment_qty	INT	実績出庫数（自動計算）

ScrapRecord（仕損記録）

工程で発生した仕損を記録するテーブル。

フィールド名	型	説明
plan_date	DATE	計画日（生産計画との紐付け）
process	FK	仕損発生工程
line	FK	仕損発生ライン
product	FK	仕損製品
qty	DECIMAL	仕損数量

フィールド名	型	説明
disposition_status	VARCHAR	判定ステータス (REJECTED=仕損確定)
recorded_at	DATETIME	記録日時

ScrapRecordDetail（仕損記録明細）

後工程仕損を前工程に展開した明細。

フィールド名	型	説明
scrap_record	FK	仕損記録
line_id	INT	展開先ライン
process_id	INT	展開先工程
product	FK	展開先製品
deduct_qty	DECIMAL	減算数量（BOM展開後）

計算ロジック

1. 仕損数の集計 (aggregate_scrap_to_backlog)

処理フロー

- 対象期間のLineBacklog.scrap_qtyをゼロリセット
- 自工程仕損を集計
 - ScrapRecordから`disposition_status = 'REJECTED'`のレコードを抽出
 - `plan_date`, `product`, `line`, `process`が一致するLineBacklogに加算
- 後工程仕損の展開分を集計
 - ScrapRecordDetailから仕損確定レコードを抽出
 - `plan_date`, `product`, `line_id`, `process_id`が一致するLineBacklogに加算

SQL イメージ

```
# 自工程仕損
LineBacklog.objects.filter(
    plan_date=scrap.plan_date,
    product=scrap.product,
    line=scrap.line,
    process=scrap.process
).update(scrap_qty=F('scrap_qty') + scrap.qty)

# 後工程仕損展開
LineBacklog.objects.filter(
    plan_date=detail.scrap_record.plan_date,
    product=detail.product,
    line_id=detail.line_id,
```

```
process_id=detail.process_id
).update(scrap_qty=F('scrap_qty') + detail.deduct_qty)
```

2. 実績出庫数の計算 (calculate_actual_shipment)

後工程の実績生産数からBOMを使って逆算。

注意: 本ロジックは最終品以外（BOM上に親が存在する製品）の出庫算出に使用する。

最終品（`product.is_final_product = true`）は顧客確定を出庫として扱う（実在庫の計算を参照）。

処理フロー

1. 対象製品を子製品として使用しているBOMItemを検索
2. 各BOMの親製品（後工程製品）のLineBacklogを検索
3. 後工程の実績生産数 × BOM数量 = 実績出庫数
4. 複数の後工程がある場合は合計

計算式

$$\text{実績出庫数} = \sum (\text{後工程}i.\text{actual_qty} \times \text{BOM}i.\text{quantity})$$

コード例

```
parent_bom_items = BOMItem.objects.filter(
    child_product=backlog.product,
    bom__is_active=True
)

for bom_item in parent_bom_items:
    downstream_backlogs = LineBacklog.objects.filter(
        product=bom_item.bom.parent_product,
        plan_date=backlog.plan_date
    )
    for downstream in downstream_backlogs:
        if downstream.actual_qty > 0:
            shipment = downstream.actual_qty * bom_item.quantity
            total_shipment += shipment
```

3. 実在庫の計算 (recalculate_stock_qty)

日次で実在庫を再計算。

計算式

$$\text{当日実在庫} = \text{前営業日実在庫} + \text{実績生産} - \text{出庫} + \text{調整} - \text{仕損}$$

処理フロー

1. 製品・ライン・期間でLineBacklogを日付順に取得
2. 各日について：
 - 前営業日在庫を取得（カレンダーが無い場合は暦日）
 - 出庫の扱いは「最終品 / 最終品以外」で分岐
（実在庫は過去～今日までを対象とし、内示・計画は出庫に使わない）
 - 最終品 (`product.is_final_product = true`)
 - `plan_date <= 今日`: 出庫 = 顧客確定 (firm)
 - `plan_date > 今日`: 出庫 = 0 (内示・計画は計上しない)
 - 最終品以外
 - `plan_date <= 今日`: 出庫 = $\Sigma(\text{親製品の実績} \times \text{BOM使用数})$
 - `plan_date > 今日`: 出庫 = 0 (内示・計画は計上しない)
 - 親が複数ある場合は合算
 - 実在庫を計算して保存
3. 前営業日のLineBacklogが無い場合は直前に計算した値を保持（0に戻さない）

コード例

```
backlog.stock_qty = (
  prev_stock
  + (backlog.actual_qty or 0)
  - actual_shipment
  + (backlog.adjust_qty or 0)
  - (backlog.scrap_qty or 0)
)
```

4. 計画在庫の計算 (recalculate_planned_stock_qty)

時制を考慮して計画在庫を再計算。

出庫の定義（計画在庫）

- 最終品（親なし）は顧客確定を出庫として扱う（内示は使用しない）
- 最終品以外はBOM上の親製品すべての合算
 - 過去 (`plan_date < 今日`) : 出庫 = $\Sigma(\text{親製品の実績} \times \text{BOM使用数})$
 - 今日 (`plan_date == 今日`) : 出庫 = $\Sigma(\text{親製品の計画} \times \text{BOM使用数})$
※ 親製品に実績があり、かつ実績 > 計画の場合は実績を使用
 - 明日以降 (`plan_date > 今日`) : 出庫 = $\Sigma(\text{親製品の計画} \times \text{BOM使用数})$

時制による分岐

過去 (`plan_date < 今日`)

実績ベースで計算。

計画在庫 = 前営業日計画在庫 + 実績生産 - 出庫 + 調整 - 仕損

今日 (plan_date == 今日)

計画ベースで計算。出庫は上記のルールに従う。

計画在庫 = 前営業日計画在庫 + 計画生産 - 出庫 + 調整

明日以降 (plan_date > 今日)

計画ベースで計算。仕損は含めない（予測不可）。

計画在庫 = 前営業日計画在庫 + 計画生産 - 出庫 + 調整

コード例

```
today = datetime.now().date()

if backlog.plan_date < today:
    # 過去：出庫 = 親実績 × 使用数（合算）
    backlog.planned_stock_qty = (
        prev_planned
        + (backlog.actual_qty or 0)
        - actual_shipment
        + (backlog.adjust_qty or 0)
        - (backlog.scrap_qty or 0)
    )
elif backlog.plan_date == today:
    # 今日：出庫 = 親計画（実績>計画なら実績）× 使用数（合算）
    backlog.planned_stock_qty = (
        prev_planned
        + (backlog.plan_qty or 0)
        - planned_shipment
        + (backlog.adjust_qty or 0)
    )
else:
    # 明日以降：出庫 = 親計画 × 使用数（合算）
    backlog.planned_stock_qty = (
        prev_planned
        + (backlog.plan_qty or 0)
        - planned_shipment
        + (backlog.adjust_qty or 0)
    )
```

5. ライン全体の在庫再計算 (recalculate_inventory_for_line)

指定ラインの全製品について在庫を再計算。

処理フロー

1. 仕損数を集計 (aggregate_scrap_to_backlog)
2. ライン・期間内の全製品IDを取得
3. 各製品について：
 - 実在庫を計算 (recalculate_stock_qty)
 - 計画在庫を計算 (recalculate_planned_stock_qty)

API仕様

POST /api/line-backlogs/recalculate_inventory/

指定ラインの在庫を再計算する。

リクエスト

```
{
  "line_id": 8,
  "start_date": "2025-12-20",
  "end_date": "2025-12-31"
}
```

レスポンス

```
{
  "detail": "Inventory recalculated successfully"
}
```

処理内容

1. パラメータバリデーション
2. `recalculate_inventory_for_line(line_id, start_date, end_date)` を実行
3. 成功レスポンスを返却

フロントエンド仕様

在庫一覧画面 (ProductionInventory.vue)

表示項目

項目名	データソース	説明
-----	--------	----

項目名	データソース	説明
内示	order_qty	後工程需要（受注数）
確定	actual_qty	実績生産数
計画	plan_qty	計画生産数
調整	adjust_qty	調整数
仕損	scrap_qty	仕損数（自動集計）
在庫	stock_qty	実在庫
計画在庫	planned_stock_qty	計画在庫

表示ルール

- **0値**: 空白表示
- **マイナス値**: 赤背景、赤文字、太字で強調
- **プラス値**: 通常表示（カンマ区切り）

在庫再計算ボタン

1. 確認ダイアログを表示
2. 表示中の全ラインIDを取得
3. 各ラインに対して`recalculateInventory` APIを呼び出し
4. 完了後、データを再読み込み

```
const recalculateInventory = async () => {
  if (!confirm('在庫と計画在庫を再計算しますか？\n※全ラインの在庫データが更新されます。')) {
    return;
  }

  const start = columns.value[0];
  const end = columns.value[columns.value.length - 1];
  const lines = [...new Set(demands.value.map(d => d.line).filter(Boolean))];

  for (const lineId of lines) {
    await api.lineBacklogs.recalculateInventory({
      line_id: lineId,
      start_date: start,
      end_date: end
    });
  }

  await load();
};
```

内示（order_qty）の更新

在庫一覧の読み込み時に、表示中のラインに対してline-backlogs/pickupを呼び出し、後工程計画からorder_qtyを再計算して反映する。

使用例

シナリオ1: 過去で実績なしの場合

前提条件:

- 12/24 (過去日)
- order_qty = 6 (後工程需要あり)
- plan_qty = 0
- actual_qty = 0 (生産しなかった)
- 前日計画在庫 = 0

計算結果:

$$\text{計画在庫} = 0 + 0 - 6 + 0 - 0 = -6$$

→ 生産しなかったが需要があったため、在庫不足 (-6)

シナリオ2: 過去で実績ありの場合

前提条件:

- 12/24 (過去日)
- order_qty = 6
- plan_qty = 10
- actual_qty = 8 (実績入力済み)
- 前日計画在庫 = 0
- 実績出庫 = 6 (後工程で実際に使用)

計算結果:

$$\text{計画在庫} = 0 + 8 - 6 + 0 - 0 = 2$$

→ 8個生産、6個出庫、2個在庫

シナリオ3: 仕損発生の場合

前提条件:

- 12/25 (過去日)
- actual_qty = 10
- 実績出庫 = 5
- scrap_qty = 2 (仕損発生)
- 前日計画在庫 = 2

計算結果:
$$\text{計画在庫} = 2 + 10 - 5 + 0 - 2 = 5$$

→ 仕損分を差し引いた在庫

運用フロー

日次運用

1. 生産実績入力

- 各ラインで実績数量 (actual_qty) を入力
- 仕損発生時はScrapRecordに記録

2. 在庫再計算

- 在庫一覧画面で「在庫再計算」ボタンをクリック
- 全ラインの在庫・計画在庫が自動更新

3. 在庫確認

- マイナス在庫（赤字表示）をチェック
- 必要に応じて調整数 (adjust_qty) を手入力

月次運用

1. 棚卸

- 実在庫とシステム在庫を比較
- 差異をadjust_qtyに入力

2. 在庫再計算

- 調整後の在庫を再計算

注意事項

計算順序

1. 仕損数の集計
2. 実在庫の計算（製品ごと、日付順）
3. 計画在庫の計算（製品ごと、日付順）

重要: 日付順に計算する必要がある（前日在庫を参照するため）

前日参照の定義

- 前日は「カレンダーの前営業日」を使用する
- ラインにカレンダーが無い場合はtiera_mukeを使用
- カレンダーが無い場合は暦日で前日を扱う

- 前営業日のLineBacklogが無い場合は直前に計算した値を保持する

後工程実績の考慮

- 実績出庫数は後工程の実績から自動計算（最終品以外）
- 後工程に複数のラインがある場合は合算
- 最終品（`product.is_final_product = true`）の出庫は顧客確定を使用
- 実在庫の出庫は過去～今日のみ計上し、内示・計画は使用しない

時制判定

- 今日の日付は`datetime.now().date()`で取得
- 過去: `plan_date < today`
- 未来: `plan_date >= today`

パフォーマンス

- 製品ごとに個別計算（並列化可能）
- 大量データの場合は期間を分割して実行推奨

今後の拡張

1. リードタイムの考慮

- 現在は同日の後工程実績を参照
- 工程間リードタイムを考慮した日付調整

2. 安全在庫の管理

- 製品ごとの安全在庫設定
- 在庫アラート機能

3. 在庫推移予測

- 受注予測に基づく将来在庫シミュレーション
- 発注点・発注量の提案

4. バッチ処理化

- 夜間バッチでの自動在庫計算
- 差分更新による高速化